

torch.sparse_compressed_tensor#

https://pytorch.org/docs/stable/generated/torch.sparse_compressed_tensor.html

Description:

Constructs a sparse tensor in Compressed Sparse format - CSR, CSC, BSR, or BSC - with specified values at the given compressed_indices and plain_indices. Sparse matrix multiplication operations in Compressed Sparse format are typically faster than that for sparse tensors in COO format. Make you have a look at the note on the data type of the indices. If the device argument is not specified the device of the given values and indices tensor(s) must match. If, however, the argument is specified the input Tensors will be converted to the given device and in turn determine the device of the constructed sparse tensor. compressed_indices (array_like) – (B+1)-dimensional array of size (*batchsize, compressed_dim_size + 1). The last element of each batch is the number of non-zero elements or blocks. This tensor encodes the index in values and plain_indices depending on where the given compressed dimension (row or column) starts. Each successive number in the tensor subtracted by the number before it denotes the number of elements or blocks in a given compressed dimension. plain_indices (array_like) – Plain dimension (column or row) coordinates of each element or block in values. (B+1)-d

Function Signature

```
torch.sparse_compressed_tensor(compressed_indices,  
plain_indices, values, size=None, *, dtype=None,  
layout=None, device=None, pin_memory=False,  
requires_grad=False, check_invariants=None) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

`dtype` (`torch.dtype`, optional) – the desired data type of returned tensor. Default: if None, infers data type from values. `layout` (`torch.layout`, required) – the desired layout of returned tensor: `torch.sparse_csr`, `torch.sparse_csc`, `torch.sparse_bsr`, or `torch.sparse_bsc`. `device` (`torch.device`, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see `torch.set_default_device()`). `device` will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types. `pin_memory` (bool, optional) – If set, returned tensor would be allocated in the pinned memory. Works only for CPU tensors. Default: False. `requires_grad` (bool, optional) – If autograd should record operations on the returned tensor. Default: False. `check_i`

Code Examples

```
>>> compressed_indices = [0, 2, 4]
>>> plain_indices = [0, 1, 0, 1]
>>> values = [1, 2, 3, 4]
>>>
torch.sparse_compressed_tensor(torch.tensor(compressed_indices,
dtype=torch.int64),
...                             torch.tensor(plain_indices,
dtype=torch.int64),
...                             torch.tensor(values),
dtype=torch.double, layout=torch.sparse_csr)
tensor(crow_indices=tensor([0, 2, 4]),
      col_indices=tensor([0, 1, 0, 1]),
      values=tensor([1., 2., 3., 4.]), size=(2, 2), nnz=4,
      dtype=torch.float64, layout=torch.sparse_csr)
```

Notes & Warnings

NOTE

Note If the device argument is not specified the device of the given values and indices tensor(s) must match. If, however, the argument is specified the input Tensors will be converted to the given device and in turn determine the device of the constructed sparse tensor.

Detailed Documentation

Documentation

Constructs a sparse tensor in Compressed Sparse format - CSR, CSC, BSR, or BSC - with specified values at the given `compressed_indices` and `plain_indices`. Sparse matrix multiplication operations in Compressed Sparse format are typically faster than that for sparse tensors in COO format. Make you have a look at the note on the data type of the indices.

If the device argument is not specified the device of the given values and indices tensor(s) must match. If, however, the argument is specified the input Tensors will be converted to the given device and in turn determine the device of the constructed sparse tensor.

`compressed_indices` (array_like) – (B+1)-dimensional array of size (*batchsize, `compressed_dim_size` + 1). The last element of each batch is the number of non-zero elements or blocks. This tensor encodes the index in values and `plain_indices` depending on where the given compressed dimension (row or column) starts. Each successive number in the tensor subtracted by the number before it denotes the number of elements or blocks in a given compressed dimension.

`plain_indices` (array_like) – Plain dimension (column or row) coordinates of each element or block in values. (B+1)-dimensional tensor with the same length as values.

`values` (array_list) – Initial values for the tensor. Can be a list, tuple, NumPy ndarray, scalar, and other types. that represents a (1+K)-dimensional (for CSR and CSC layouts) or (1+2+K)-dimensional tensor (for BSR and BSC layouts) where K is the number of dense dimensions.

`size` (list, tuple, torch.Size, optional) – Size of the sparse tensor: (*batchsize, nrows * blocksize[0], ncols * blocksize[1], *densesize) where blocksize[0] == blocksize[1] == 1 for CSR and CSC formats. If not provided, the size will be inferred as the minimum size

big enough to hold all non-zero elements or blocks.

`dtype` (`torch.dtype`, optional) – the desired data type of returned tensor. Default: if None, infers data type from values.

`layout` (`torch.layout`, required) – the desired layout of returned tensor: `torch.sparse_csr`, `torch.sparse_csc`, `torch.sparse_bsr`, or `torch.sparse_bsc`.

`device` (`torch.device`, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see `torch.set_default_device()`). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types.

`pin_memory` (bool, optional) – If set, returned tensor would be allocated in the pinned memory. Works only for CPU tensors. Default: False.

`requires_grad` (bool, optional) – If autograd should record operations on the returned tensor. Default: False.

`check_invariants` (bool, optional) – If sparse tensor invariants are checked. Default: as returned by `torch.sparse.check_sparse_tensor_invariants.is_enabled()`, initially False.